

experiment provides remarkable evidence that most of the memorization effectively happens in the deep representation and not in the last layer.

1.2 Related Work

For a more detailed comparison of the long tail theory with the standard approaches to understanding of generalization and work on interpolating methods we refer the reader to [Fel19].

Memorization of data has been investigated in the context of privacy-preserving ML. Starting from [SSSS17], multiple works have demonstrated that the output of the trained neural network can be used to perform successful membership inference attacks, that is to infer with high accuracy whether a given data point is part of the training set. An important problem in this area is to find learning algorithms that are more resistant to such attacks on privacy. Our results suggest that reducing memorization will also affect the accuracy of learning.

Arpit et al. [Arp+17] examine the relationship between memorization of random labels and performance of the network on true labels. The work demonstrates that using various regularization techniques, it is possible to reduce the ability of the training algorithm to fit random labels without significantly impacting its test accuracy on true labels. The explanation proposed for this finding is that memorization is not necessary for learning. However memorization is used informally to refer to fitting the entire randomly labeled dataset. Even with regularization, the algorithms used in this work memorize a significant fraction of randomly labeled examples and fit the true training data (nearly) perfectly.

Carlini et al. [CEP19] consider different ways to measure how “prototypical” each of the data points is according to several metrics and across multiple datasets. They examine 5 different metrics and draw a variety of insights about the metrics from the visualisations of rankings along these metrics. They also discuss “memorized exceptions”, “uncommon submodes” and outliers informally. While the memorization value we investigate also identifies atypical examples and outliers, it is not directly related to these metrics. This work also briefly reports on an unsuccessful attempt to find individual training examples that influence individual test examples on MNIST. As our experiments demonstrate, such high-influence pairs are present in MNIST and therefore this result confirms that finding them via the direct leave-one-out method while ensuring statistical significance is computationally infeasible even on MNIST.

The use of random data subsamples is standard in data analysis, most notably in bagging, bootstrapping and cross validation. In these applications the results from random subsamples are aggregated to estimate the properties of the results of data analysis as a whole whereas we focus on the properties of individual samples. Concurrent work of Jiang et al. [JZTM20] uses data subsamples to estimate the regularity (or easiness) of each training example. This score (referred to as the *empirical consistency score*) coincides with the second term in our subsampled memorization value estimator (Alg. 1, line 5). The value of this estimator is typically equal to one minus our memorization estimate since fitting hard-to-predict training examples requires memorizing their labels. The score was derived independently of our work and its computation in [JZTM20] builds on the experimental framework developed in this work. The focus in [JZTM20] is on the investigation of several proxies for the consistency score and their experimental results are otherwise unrelated to ours.

Toneva et al. [TSCTBG19] investigate a “forgetting” phenomenon in which an example that was predicted correctly at some point during training becomes misclassified. They empirically demonstrate that examples that are never “forgotten” tend to have low marginal utility. The notion of memorization we consider is not directly related to such “forgetting”.

2 Estimation and Selection Procedures

In this section we describe how our memorization and influence estimators are defined and computed. We also describe the selection criteria for high-influence pairs of examples.

Memorization and influence estimators: Our goal is to measure the label memorization by an algorithm $\mathcal{A}$ on a (training) dataset S and example (x_i, y_i) (eq. (1)) and the influence of a training example (x_i, y_i) on a test example (x'_j, y'_j) . Both of these values are a special case of measuring the influence of (x_i, y_i) on the expected accuracy at some example $z = (x, y)$ or

$$\text{infl}(\mathcal{A}, S, i, z) := \Pr_{h \leftarrow \mathcal{A}(S)}[h(x) = y] - \Pr_{h \leftarrow \mathcal{A}(S \setminus i)}[h(x) = y]. \quad (3)$$

Clearly, $\text{mem}(\mathcal{A}, S, i) = \text{infl}(\mathcal{A}, S, i, (x_i, y_i))$, that is memorization corresponds to the influence of example i on the accuracy on itself (or self-influence).

As discussed in Sec. 1.1, directly estimating the influence of all n training examples within standard deviation σ requires training on the order of n/σ^2 models. Thus we propose a closely-related influence value that looks at the expected influence of an example (x_i, y_i) relative to a dataset that includes a random subset of S of size $m < n$. More formally, for a set of indices $I \subseteq [n]$ ($[n]$ is defined as the set $\{1, \dots, n\}$), let $S_I = (x_i, y_i)_{i \in I}$ be the dataset consisting of examples from S with indices in I . For a set of indices $J \subseteq [n]$, let $P(J, m)$ denote the uniform distribution over all subsets of J of size m . Then we define:

$$\text{infl}_m(\mathcal{A}, S, i, z) := \mathbf{E}_{I \sim P([n] \setminus \{i\}, m-1)} [\text{infl}(\mathcal{A}, S_{I \cup \{i\}}, i, z)],$$

where by sampling a random subset of size $m - 1$ that excludes index i we ensure that $S_{I \cup \{i\}}$ is uniform over all subsets of size m that include the index i .

We now show that subsampled influence values can be estimated with standard deviation σ by training just $O(1/\sigma^2)$ models.

Lemma 2.1. *There exists an algorithm that for every dataset $S \in (X \times Y)^n$, learning algorithm $\mathcal{A}$, $m \in [n]$ and integer t , runs $\mathcal{A}$ t times and outputs estimates $(\mu_i)_{i \in [n]}$ such that for every $i \in [n]$ and $p = \min(m/n, 1 - m/n)$,*

$$\mathbf{E} [(\text{infl}_m(\mathcal{A}, S, i, z) - \mu_i)^2] \leq \frac{1}{pt} + \frac{1}{(1-p)t} + \frac{e^{-pt/16}}{2},$$

where the expectation is with respect to the randomness of $\mathcal{A}$ and the randomness of the estimation algorithm.

We include the proof in Sec. A. The estimator exploits the fact that by training models on random subsets of size m we will, with high probability, obtain many models trained on subsets that include index i for every i and also many subsets that exclude i . By linearity of expectation, this gives an estimate of $\text{infl}_m(\mathcal{A}, S, i, z)$. Alg. 1 describes the resulting algorithm for estimating all the memorization and influence values. We use $k \sim [t]$ to denote index k being chosen randomly and uniformly from the set $[t]$ (the probabilities for such sampling are computed by enumerating over all values of k).

Algorithm 1 Memorization and influence value estimators

Require: Training dataset: $S = ((x_1, y_1), \dots, (x_n, y_n))$, testing dataset $S_{\text{test}} = ((x'_1, y'_1), \dots, (x'_{n'}, y'_{n'}))$, learning algorithm $\mathcal{A}$, subset size m , number of trials t .

- 1: Sample t random subsets of $[n]$ of size m : $I_1, I_2, \dots, I_t$.
 - 2: **for** $k = 1$ to t **do**
 - 3: Train model h_k by running $\mathcal{A}$ on S_{I_k} .
 - 4: **for** $i = 1$ to n **do**
 - 5: $\widetilde{\text{mem}}_m(\mathcal{A}, S, i) := \Pr_{k \sim [t]}[h_k(x_i) = y_i \mid i \in I_k] - \Pr_{k \sim [t]}[h_k(x_i) = y_i \mid i \notin I_k]$.
 - 6: **for** $j = 1$ to n' **do**
 - 7: $\widetilde{\text{infl}}_m(\mathcal{A}, S, i, j) := \Pr_{k \sim [t]}[h_k(x'_j) = y'_j \mid i \in I_k] - \Pr_{k \sim [t]}[h_k(x'_j) = y'_j \mid i \notin I_k]$.
 - 8: **return** $\widetilde{\text{mem}}_m(\mathcal{A}, S, i)$ for all $i \in [n]$; $\widetilde{\text{infl}}_m(\mathcal{A}, S, i, j)$ for all $i \in [n], j \in [n']$.
-

The larger the subset size parameter m , the closer is our estimator to the original $\text{infl}(\mathcal{A}, S, i, z)$. At the same time, we need to ensure that we have a sufficient number of random datasets that exclude each example (x_i, y_i) . To roughly balance these requirements in all of our experiments we chose $m = 0.7 \cdot n$. Due to computational constraints we chose the number of trials to be $t = 2000$ for ImageNet and $t = 4000$ for MNIST/CIFAR-100.

We remark that for $m = n/2$ our influence value is closely-related to the Shapley value of example (x_i, y_i) for the function that measures the expected accuracy of the model on point z . This follows from the fact that for functions that

are symmetric (do not depend on the order of examples) the Shapley value is equal to the expected marginal utility of example i relative to the random and uniform subset of all examples. For sufficiently large n , such subsets have size close to $n/2$ with high probability. The Shapley value is itself a natural and well-studied measure of the contribution of each point to the value of a function and thus provides an additional justification for the use of our subsampled influence values.

Selection of high-influence pairs of examples: To find the high-influence pairs of examples we select all pairs of examples $(x_i, y_i) \in S$ and $(x'_j, y'_j) \in S_{test}$ for which $\widehat{\text{mem}}_m(\mathcal{A}, S, i) \geq \theta_{\text{mem}}$; $\widehat{\text{infl}}_m(\mathcal{A}, S, i, j) \geq \theta_{\text{infl}}$ and $y_i = y'_j$. The last condition is used since the long tail theory only explains improvement in accuracy from examples in the same subpopulation and allows to reduce the noise in the selected estimates.

Selection of pairs that is based on random estimates introduces selection bias into the estimates of values that are close to the threshold value. The choice of θ_{mem} is less important and is almost unaffected by bias due to a relatively small set of estimates. We have chosen $\theta_{\text{mem}} = 0.25$ as a significant level of memorization. In choosing θ_{infl} we wanted to ensure that the effect of this selection bias is relatively small. To measure this effect we ran our selection procedure with various thresholds on CIFAR-100 twice, each based on 2000 trials. We chose $\theta_{\text{infl}} = 0.15$ as a value for which the Jaccard similarity coefficient between the two selected sets is ≥ 0.7 . In these two runs 1095 and 1062 pairs were selected, respectively with $\approx 82\%$ of the pairs in each set also appearing in the other set. We have used the same thresholds for all three datasets for consistency. More details on the consistency of our selection process can be found in Sec. 3.5.

3 Empirical Results

In this section we describe the results of the experiments based on the methods and parameters specified in Sec. 2. We use ResNet50 in both ImageNet and CIFAR-100 experiments, which is a Residual Network architecture widely used in the computer vision community [HZRS16]. Full details of the experimental setup and training algorithms are given in Sec. B.

3.1 Examples of memorization value estimates

In Fig. 1 we show examples of the estimated subsampled memorization values around 0, 0.5, and 1, respectively. Additional examples can be found at [FZ20]. These examples suggest that the estimates reflect our intuitive interpretation of label memorization. In particular, some of the examples with estimated memorization value close to 0 are clearly typical whereas those with value close to 1 are atypical, highly ambiguous or mislabeled.

3.2 Marginal utility of memorized examples

Fig. 2 demonstrates the significant effect that the removal of memorized examples from the dataset has on the test set accuracy. One could ask whether this effect is solely due to the reduction in number of available training examples as a result of the removal. To answer this question we include in the comparison the accuracy of the models trained on the identical number of examples which are chosen randomly from the entire dataset. Remarkably, memorized examples have higher marginal utility than the identical number of randomly chosen examples. The likely reason for this is that most of the randomly chosen examples are easy and have no marginal utility.

3.3 Estimation of influence and marginal utility of high-influence examples

We compute the estimated influences and select high-influence pairs of examples as described in Sec. 2. Overall we found 35/1015/1641 pairs in MNIST/CIFAR-100/ImageNet. In Fig. 3 we give histograms of the number of such pairs for every level of influence. The number of unique test examples in these pairs is 33/888/1462 (comprising 0.33%/8.88%/2.92% of the test set). Of those 31/774/1298 are influenced (above the 0.15 threshold) by a single training example. On CIFAR-100 and ImageNet this confirms the importance of the subpopulations in the long tail that have unique representatives for the generalization error. The results on the MNIST are consistent with the fact that it is a